

Runbook: S3 Replica Path (ADR-0002 Option C)

Point-in-time physical replica of a GCS-resident Iceberg table into S3, read intra-region by Snowflake — zero per-query egress; the cross-cloud cost is paid once per sync. Use for tables that fail the ADR-0006 break-even test (monthly scanned bytes $\gg$ table size) or for S3-only consumers (Athena, Redshift). The default path for everything else is [runbook-zero-copy.md](#).

Executed successfully 2026-07-16 against a Snowflake AWS us-east-2 trial and a personal AWS account.

0. Prerequisites

- The zero-copy path already working (this path reuses the GCP catalog and the Dataproc pattern from it).
- Your own AWS account (the Snowflake account *runs on* AWS, but the bucket and IAM role live in an account you control).
- An IAM user with programmatic access and sufficient rights (S3 + IAM; AdministratorAccess is fine on a personal account — scope down for shared ones). Credentials via the standard chain:
`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_DEFAULT_REGION=us-east-2` (match the Snowflake account region so replica reads are intra-region).
- boto3 locally (`pip install boto3`).

1. AWS: bucket + role (one-time)

```
python scripts/11_aws_replica_setup.py create --bucket <s3-bucket>
# creates the bucket in us-east-2 + role iceberg-poc-snowflake-replica
# (read-only on that one bucket) and prints STORAGE_AWS_ROLE_ARN
```

2. Snowflake: external volume (one-time)

```
CREATE OR REPLACE EXTERNAL VOLUME s3_replica_vol
  STORAGE_LOCATIONS = ((
    NAME = 's3-replica'
    STORAGE_PROVIDER = 'S3'
    STORAGE_BASE_URL = 's3://<s3-bucket>/'
    STORAGE_AWS_ROLE_ARN = '<ARN from step 1>'
  ))
  ALLOW_WRITES = FALSE;

DESC EXTERNAL VOLUME s3_replica_vol;
-- note STORAGE_AWS_IAM_USER_ARN and STORAGE_AWS_EXTERNAL_ID
```

3. AWS: trust handshake (one-time)

```
python scripts/11_aws_replica_setup.py trust --bucket <s3-bucket> \
  --iam_user_arn '<STORAGE_AWS_IAM_USER_ARN>' --external_id '<STORAGE_AWS_EXTERNAL_ID>'
```

Scope of the connection this creates: exactly one Snowflake-side IAM user (living in Snowflake's AWS account), presenting exactly one external id, may assume a role that can only read this one bucket. No keys are exchanged; revoke by deleting the role or dropping the external volume.

4. Replicate (repeat per refresh)

```
./scripts/10_replicate_to_s3.sh <s3-bucket> shared_aws.orders
```

Two stages: a Dataproc Serverless batch runs Iceberg's `rewrite_table_path` (metadata stores **absolute paths**, so a byte-for-byte copy would still point at gs:// — the procedure stages s3://-prefixed metadata and emits a copy plan), then `s3_copy_filelist.py` executes the plan (GCS reads via gcloud ADC, S3 writes via boto3). The script prints the `METADATA_FILE_PATH` for step 5.

5. Snowflake: register + verify

```
CREATE OR REPLACE CATALOG INTEGRATION s3_replica_int
  CATALOG_SOURCE = OBJECT_STORE TABLE_FORMAT = ICEBERG ENABLED = TRUE;

CREATE OR REPLACE ICEBERG TABLE replica_orders
  EXTERNAL_VOLUME = 's3_replica_vol'
  CATALOG = 's3_replica_int'
  METADATA_FILE_PATH = '<printed by step 4>';

-- same table, both paths:
SELECT 'zero_copy' AS path, COUNT(*) FROM shared_gcp_data.shared_aws.orders
UNION ALL
SELECT 's3_replica', COUNT(*) FROM replica_orders;
```

6. Refresh cadence

The replica is **stale by design** between syncs. Refresh = rerun step 4, then:

```
ALTER ICEBERG TABLE replica_orders REFRESH '<new metadata path>';
```

Production shape: schedule step 4 (Composer / Cloud Scheduler + Cloud Run job) per table that passed the break-even review; `rewrite_table_path` supports incremental mode (start/end snapshot) so refreshes copy

only new files.

Teardown

```
DROP ICEBERG TABLE replica_orders;
DROP CATALOG INTEGRATION s3_replica_int;
DROP EXTERNAL VOLUME s3_replica_vol;

# AWS: empty + delete the bucket, delete role iceberg-poc-snowflake-replica
```

Troubleshooting

Symptom	Likely cause / fix
AccessDenied ... s3:CreateBucket	IAM user has no policy attached (a fresh user has none — attach before running)
Spark: missing '(' at '-' ' on CALL	Hyphenated catalog names must be backtick-quoted in Spark SQL (script does this)
Spark: 403 writing staging metadata	Vended credentials are downscoped to the TABLE's prefix — staging_location must live inside the table location (script derives it via DESCRIBE TABLE EXTENDED)
No URLs matched .../file-list	The copy plan is a Spark output <i>directory</i> of part-files, not one object (copier handles both)
External volume verify fails	Trust policy not updated (step 3), or external id mismatch — CREATE OR REPLACE EXTERNAL VOLUME mints a NEW external id; redo step 3 after any replace
Snowflake reads fail after refresh	ALTER ICEBERG TABLE ... REFRESH not run, or old metadata path
Replica disagrees with zero- copy table	Expected between syncs (staleness by design); rerun step 4